



Daffodil *International* **University**

ASSIGNMENT

Title: Final Assignment

Course Title: Compiler Design

Course Code: CSE313

Submitted To:

Teacher's Name: Dewan Mamun Raza

Designation: Assistant Professor

Department of Computer Science and Engineering

Faculty of Science and Information Technology

Daffodil International University

Submitted By:

Name: Supan Roy

ID: 232-15-716

Section: 65_D

Department of CSE

Daffodil International University

Submission Date: 17 December, 2025

Answer to the question no: 1

Given grammar -

$$E \rightarrow E+T \mid E-T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid id$$

Eliminating left recursion -

$$E \rightarrow E+T \mid E-T \mid T$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid -TE' \mid \epsilon$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid /FT' \mid \epsilon$$

No left recursion needed for $F \rightarrow (E) \mid id$

Answer to the question no: 2

Given grammar -

$$A \rightarrow AaB \mid Ab \mid c$$

$$B \rightarrow Bd \mid e$$

Eliminating left recursion -

$$A \rightarrow AaB \mid Ab \mid c$$

$$A \rightarrow cA'$$

$$A' \rightarrow aBA' \mid bA' \mid \epsilon$$

$$B \rightarrow Bd \mid e$$

$$B \rightarrow eB'$$

$$B' \rightarrow dB' \mid \epsilon$$

Answer to the question no: 3

Given grammar -

domain $\rightarrow$ web | website | webapp | diu.bd | edu.bd | diu.edu | edu.ac

Left factored grammar -

domain $\rightarrow$ web web'
 | diu. diu'
 | edu. edu'

web' $\rightarrow$ ϵ | site | app

diu' $\rightarrow$ bd | edu

edu' $\rightarrow$ bd | ac

Answer to the question no: 4

Given grammar -

$S \rightarrow abe | abd | aef | g$

Left factored grammar -

$S \rightarrow aS' | g$

$S' \rightarrow bS'' | ef$

$S'' \rightarrow c | d$

Answer to the question no: 5

Given grammar -

$S \rightarrow DMY$

$D \rightarrow NN$

$M \rightarrow \text{july}$

$Y \rightarrow NNNN$

$N \rightarrow 3|6|2|0|4$

Input string: 36 july 2024

LMR

$S \rightarrow DMY$

$S \rightarrow NMY \quad [D \rightarrow NN]$

$\rightarrow 3NMY \quad [N \rightarrow 3]$

$\rightarrow 36MY \quad [N \rightarrow 6]$

$\rightarrow 36 \text{ july } Y \quad [M \rightarrow \text{july}]$

$\rightarrow 36 \text{ july } NNNN \quad [Y \rightarrow NNNN]$

$\rightarrow 36 \text{ july } 2NNN \quad [N \rightarrow 2]$

$\rightarrow 36 \text{ july } 20NN \quad [N \rightarrow 0]$

$\rightarrow 36 \text{ july } 202N \quad [N \rightarrow 2]$

$\rightarrow 36 \text{ july } 2024 \quad [N \rightarrow 4]$

RMD

$S \rightarrow DMY$

$\rightarrow DM NNNN \quad [Y \rightarrow NNNN]$

$\rightarrow DM NNN4 \quad [N \rightarrow 4]$

$\rightarrow DM NN24 \quad [N \rightarrow 2]$

$\rightarrow DM N024 \quad [N \rightarrow 0]$

$\rightarrow DM 2024 \quad [N \rightarrow 2]$

$\rightarrow D \text{ july } 2024 \quad [M \rightarrow \text{july}]$

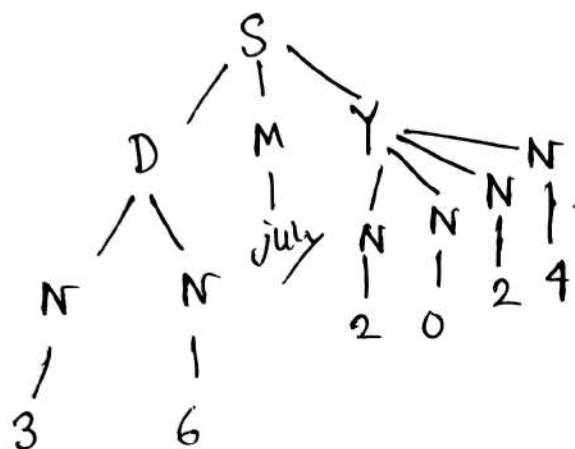
$\rightarrow NN \text{ july } 2024 \quad [D \rightarrow NN]$

$\rightarrow N6 \text{ july } 2024 \quad [N \rightarrow 6]$

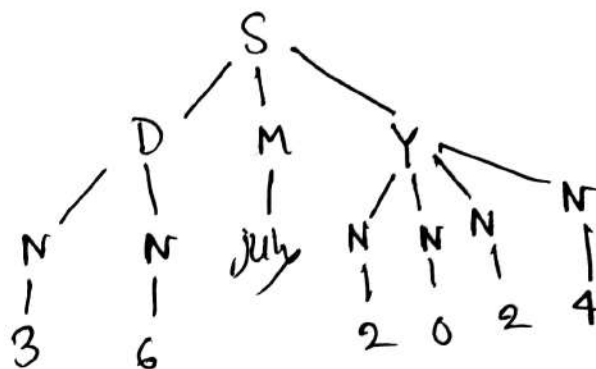
$\rightarrow 36 \text{ july } 2024 \quad [N \rightarrow 3]$

Parse Tree

For LMD



For RMD



Therefore, each non-terminal has only one valid way to derive the required part of the string and no alternative production sequences exist, the given CFG is Not ambiguous.

Answer to the question no: 6

The second statement of the given function is-

$$\text{ans} = a * x + b * x + c$$

Phases of a Compiler

⇒ Phase 1: Lexical Analysis

Input: $\text{ans} = a * x + b * x + c$

Tokenization:

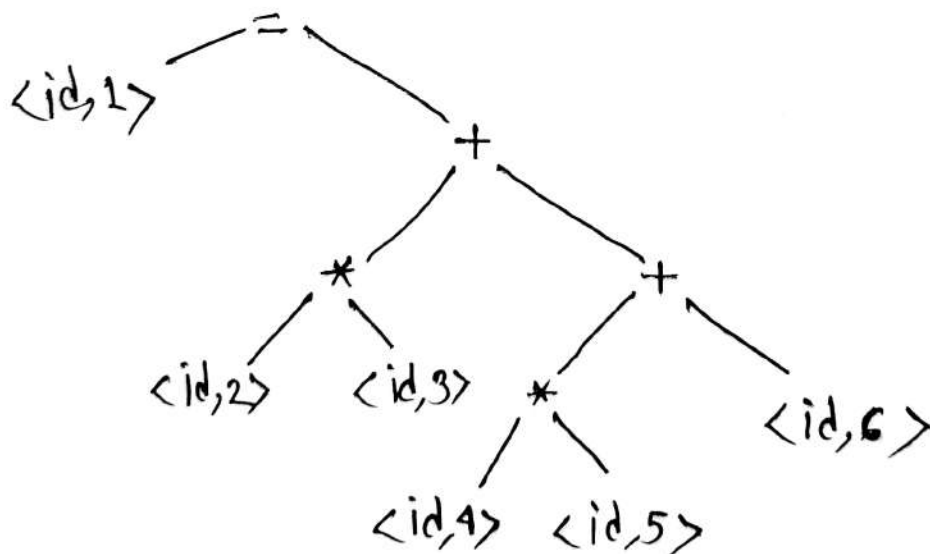
$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle * \rangle \langle \text{id}, 3 \rangle \langle + \rangle \langle \text{id}, 4 \rangle \langle * \rangle \langle \text{id}, 5 \rangle$
 $\langle + \rangle \langle \text{id}, 6 \rangle$

Symbol Table

Lexeme	Token Type
ans	identifier
=	assignment operator
a	identifier
*	multiplication operator
x	identifier
+	addition operator
b	identifier
*	multiplication operator
x	identifier
+	addition operator
c	identifier

⇒ Phase 2: Syntax Analysis

Parse tree



⇒ Phase 3: Semantic Analysis

Type checking:

$a \rightarrow \text{float}, b \rightarrow \text{float}, c \rightarrow \text{float}, x \rightarrow \text{int}$

Operations:

$a * x \rightarrow \text{float} * \text{int} \rightarrow \text{float}$

$b * x \rightarrow \text{float} * \text{int} \rightarrow \text{float}$

$(a * x + b * x + c) \rightarrow \text{float}$

$\text{ans} \rightarrow \text{float}$

So, all types are compatible and variables are declared.

⇒ Phase 4: Intermediate Code Generation

$t_1 = a * x$

$t_2 = b * x$

$t_3 = t_1 + t_2$

$t_4 = t_3 + c$

$\text{ans} = t_4$

⇒ Phase 5: Code Optimization

$t_1 = a * x$

$t_2 = b * x$

$t_3 = t_1 + t_2$

$t_4 = t_3 + c$

$t_4 = \text{ans}$

⇒ Phase 6: Target Code Generation

LOAD a
MUL x
STORE t1

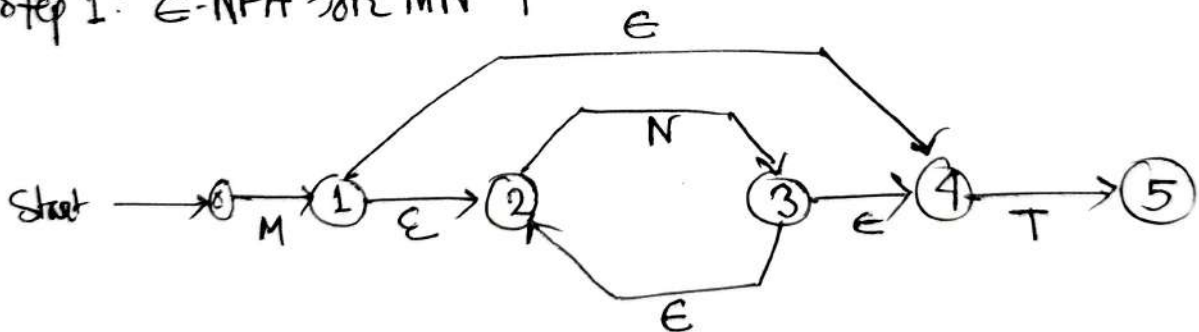
LOAD b
MUL x
ADD t1

ADD c
STORE ans

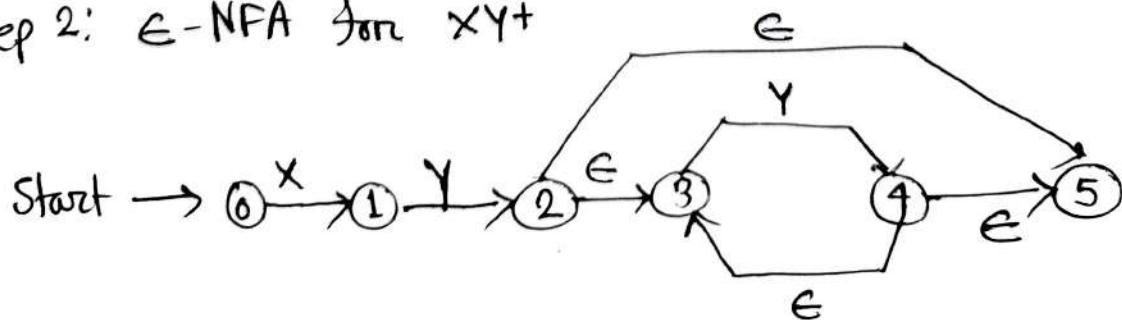
Answer to the question no: 7

① Regular Expression
 $MN^*T \mid XY^+$

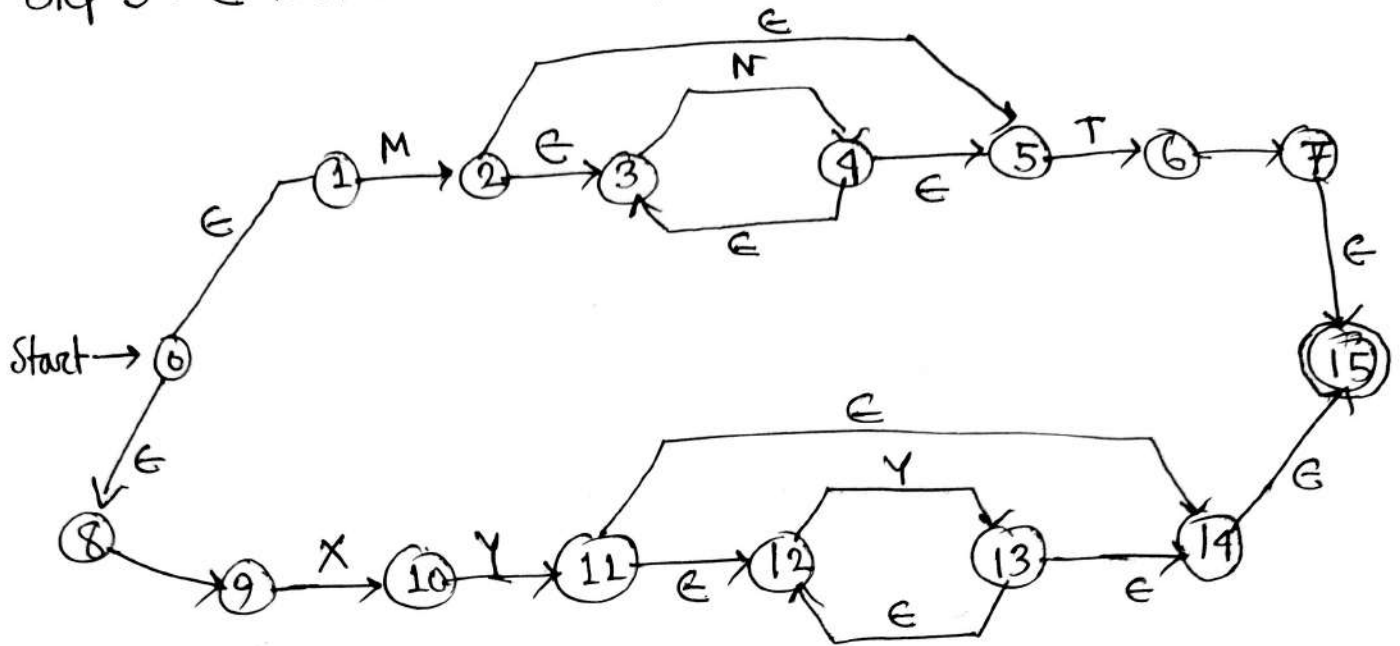
Step 1: ϵ -NFA for MN^*T



Step 2: ϵ -NFA for XY^+

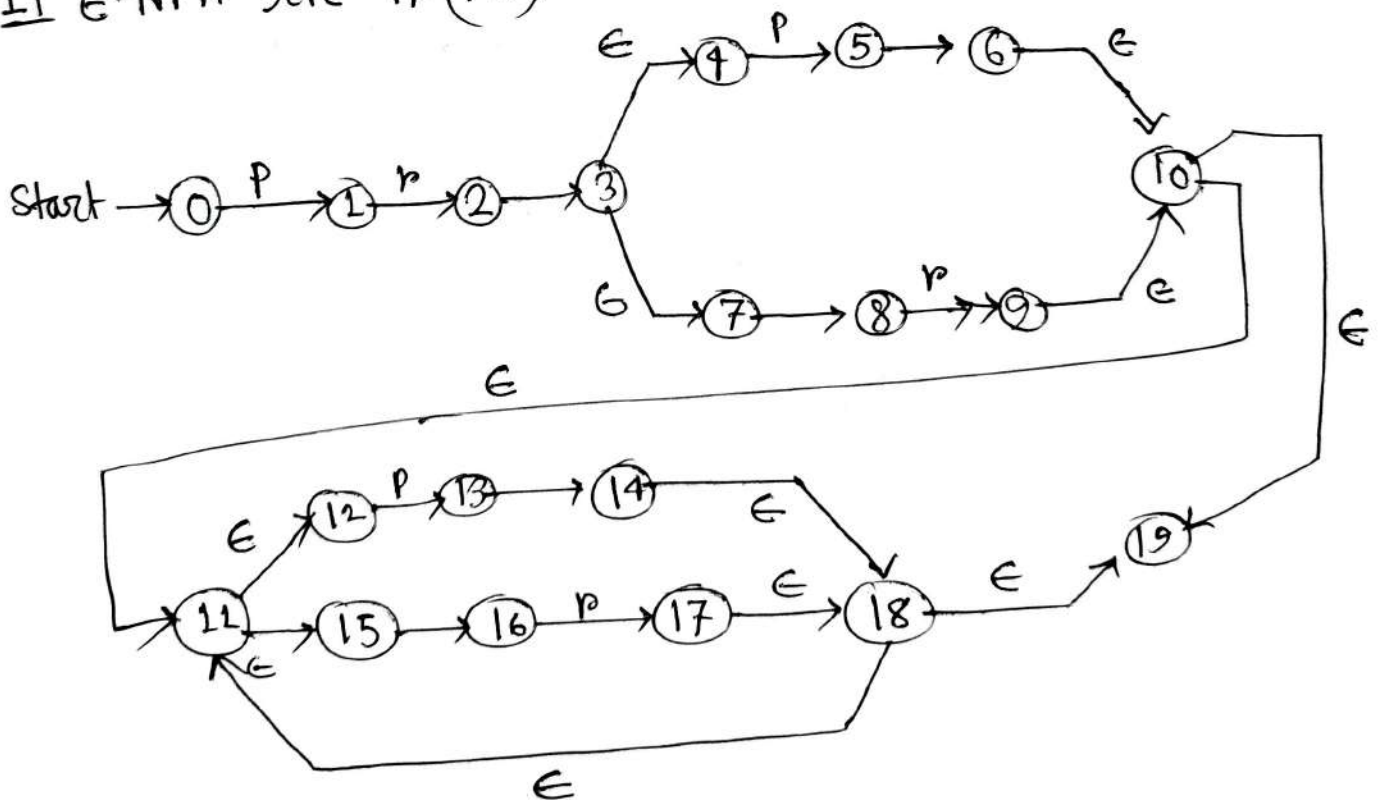


Step 3: ϵ -NFA for $MN^*T \mid XY^+$

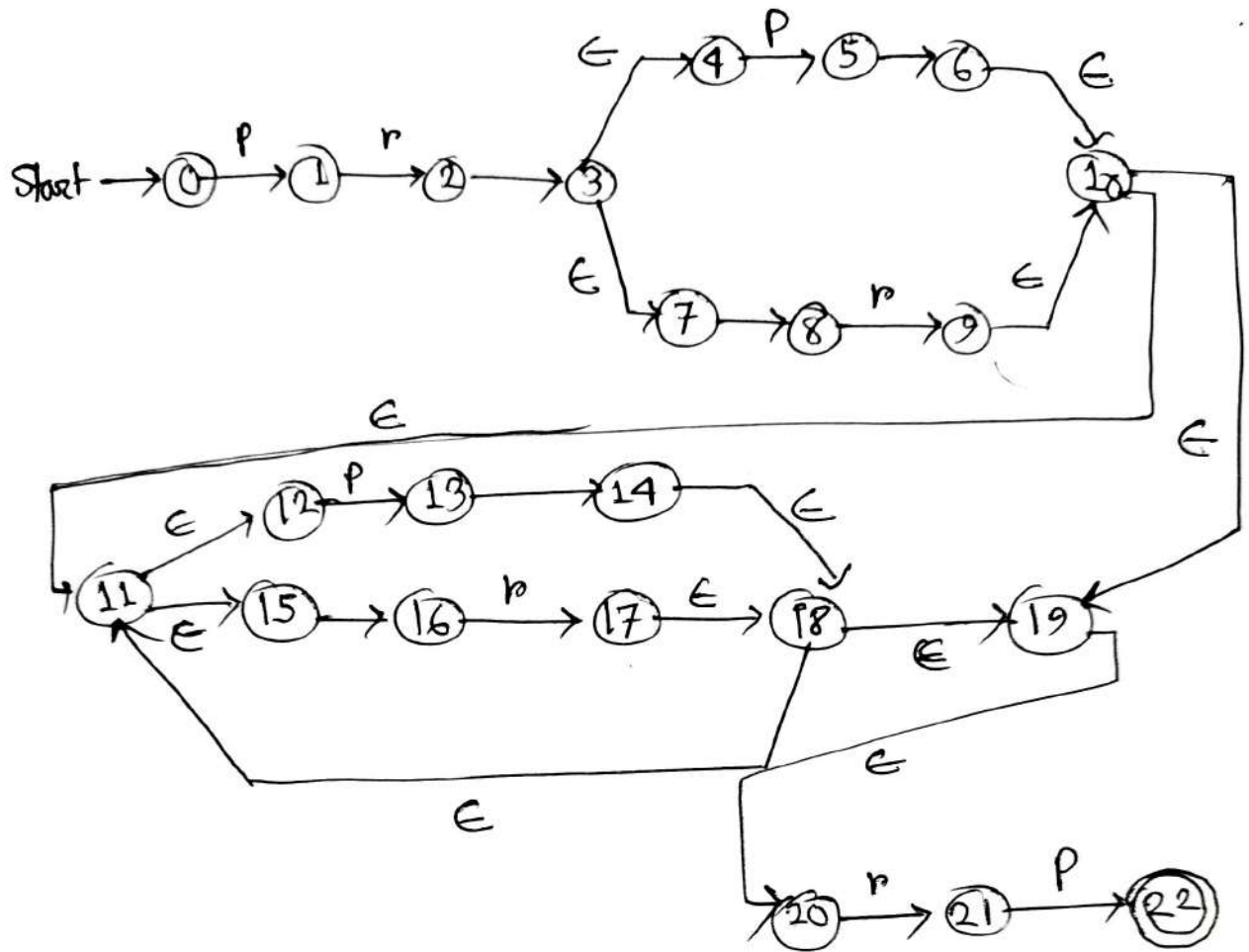


(ii) $Pr(P|r) + rP$

11 ϵ -NFA for $Pr(P|r) +$



21 ϵ -NFA for $Pp (p|p)^+ nP$



Answer to the question no: 8

Given CFG -

$$E \rightarrow E+T \mid E-T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid id$$

Eliminating left recursion -

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid -TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid /FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

First & Follow

Production	FIRST	FOLLOW
$E \rightarrow TE'$	$\{ (, id \}$	$\{), \$ \}$
$E' \rightarrow +TE' \mid -TE' \mid \epsilon$	$\{ +, -, \epsilon \}$	$\{), \$ \}$
$T \rightarrow FT'$	$\{ (, id \}$	$\{ +, -,), \$ \}$
$T' \rightarrow *FT' \mid /FT' \mid \epsilon$	$\{ *, /, \epsilon \}$	$\{ +, -,), \$ \}$
$F \rightarrow (E) \mid id$	$\{ (, id \}$	$\{ *, /, +, -,), \$ \}$

Constructing the LL(1) parsing table:

Non-Terminal	(	id	+	-	*	/	)	\$
E	$E \rightarrow TE'$	$E \rightarrow TE'$						
E'			$E' \rightarrow TE'$	$E' \rightarrow TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$	$T \rightarrow FT'$						
T'			$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow *FT$	$T' \rightarrow /FT'$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow (E)$	$F \rightarrow id$						

Answer to the question no: 9

Given CFG -

$E \rightarrow 5^*T \mid 5+T$; left factored form

$E \rightarrow 5E'$

$E' \rightarrow *T \mid +T$

$T \rightarrow MS$

$S \rightarrow QM \mid \epsilon$

$Q \rightarrow + \mid *$

$M \rightarrow a \mid b \mid c$

First & Follow

Production	FIRST	FOLLOW
$E \rightarrow 5E'$	{5}	{\$}
$E' \rightarrow *T \mid +T$	{*, +}	{\$}
$T \rightarrow MS$	{a, b, c}	{\$}
$S \rightarrow QM \mid \epsilon$	{+, *, ϵ }	{\$}
$Q \rightarrow + \mid *$	{+, *}	{a, b, c}
$M \rightarrow a \mid b \mid c$	{a, b, c}	{+, *, \$}

LL(1) Parsing Table

Non Terminal	S	a	b	c	+	*	\$
E	$E \rightarrow SE'$						
E'					$E' \rightarrow +T$	$E' \rightarrow *T$	
T		$T \rightarrow MS$	$T \rightarrow MS$	$T \rightarrow MS$			
S					$S \rightarrow QM$	$S \rightarrow QM$	$S \rightarrow \epsilon$
Q					$Q \rightarrow +$	$Q \rightarrow *$	
M		$M \rightarrow a$	$M \rightarrow b$	$M \rightarrow c$			

Answer to the question no: 10

Given CFG

Expense $\rightarrow$ Tax Salary

Salary $\rightarrow$ Grocery Tax Salary $| / | \epsilon$

Grocery $\rightarrow + | - | \epsilon$

Tax $\rightarrow$ Factor Benefit

Benefit $\rightarrow$ Cost Factor Benefit $| / | \epsilon$

Cost $\rightarrow *$

Factor $\rightarrow (Expense) | num$

First & Follow

Production / Non-Terminals	FIRST	FOLLOW
Expense	{ (, num }	{), \$ }
Salary	{ +, -, (, num, /, € }	{), \$ }
Grocery	{ +, -, € }	{ (, num }
Tax	{ (, num }	{ +, -, (, num, /, \$ }
Benefit	{ *, /, € }	{ +, -, (, num, /,), \$ }
Cost	{ * }	{ (, num }
Factor	{ (, num }	{ +, /, +, -, (, num,), \$ }

LL(1) Parsing Table

Non Terminal	(	num	+	-	*	/	)	\$
Expense	Expense $\rightarrow$ Tax Salary	Expense $\rightarrow$ Tax Salary						
Salary	Salary $\rightarrow$ Grocery Tax Salary	Salary $\rightarrow$ Grocery Tax Salary	Salary $\rightarrow$ Grocery Tax Salary	Salary $\rightarrow$ Grocery Tax Salary		Salary $\rightarrow$ /	Salary $\rightarrow$ €	Salary $\rightarrow$ €
Grocery	Grocery $\rightarrow$ €	Grocery $\rightarrow$ €	Grocery $\rightarrow$ +	Grocery $\rightarrow$ -				
Tax	Tax $\rightarrow$ Factor Benefit	Tax $\rightarrow$ Factor Benefit						
Benefit	Benefit $\rightarrow$ €	Benefit $\rightarrow$ €	Benefit $\rightarrow$ €	Benefit $\rightarrow$ €		Benefit $\rightarrow$ /	Benefit $\rightarrow$ €	Benefit $\rightarrow$ €
Cost								
Factor	Factor $\rightarrow$ (Expense)	Factor $\rightarrow$ num						

Answer to the question no: 11

Given Grammar-

$$A \rightarrow AnyB | xC$$

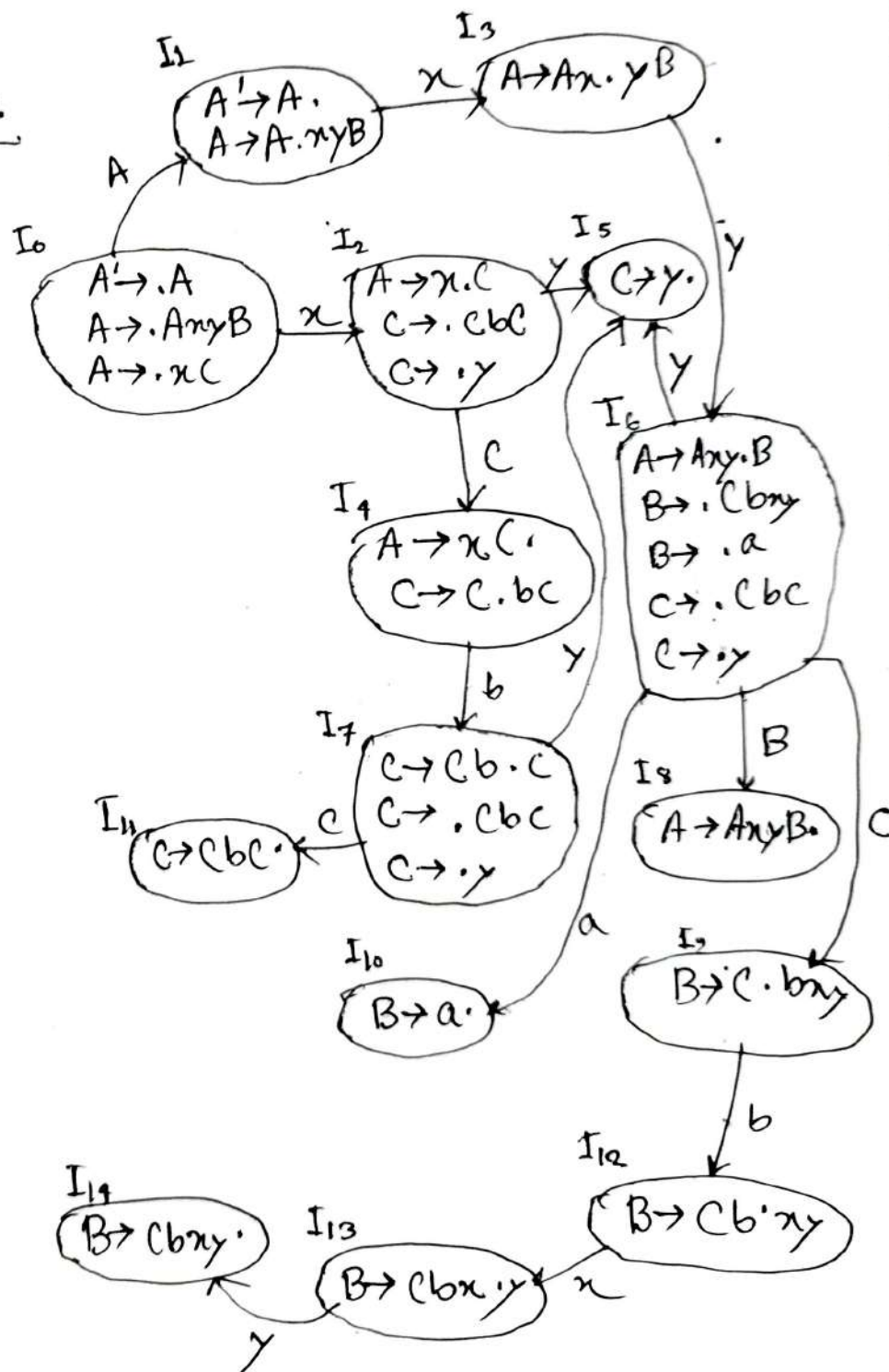
$$B \rightarrow cbxy | a$$

$$C \rightarrow cbcy$$

$$D \rightarrow a/n$$

Augmented Grammar

- (0) $A' \rightarrow A$
- (1) $A \rightarrow .AnyB$
- (2) $A \rightarrow .xC$
- (3) $B \rightarrow .cbxy$
- (4) $B \rightarrow .a$
- (5) $C \rightarrow .cbcy$
- (6) $C \rightarrow .y$



Answer to the question no: 12

Given grammar-

$$A \rightarrow A^*BA \mid xC\#y$$

$$B \rightarrow z,BC \mid CuA$$

$$C \rightarrow BvA \mid ^*zD$$

$$D \rightarrow xy,$$

Augmented grammar

(0) $A' \rightarrow A$

(1) $A \rightarrow A^*BA$

(2) $A \rightarrow xC\#y$

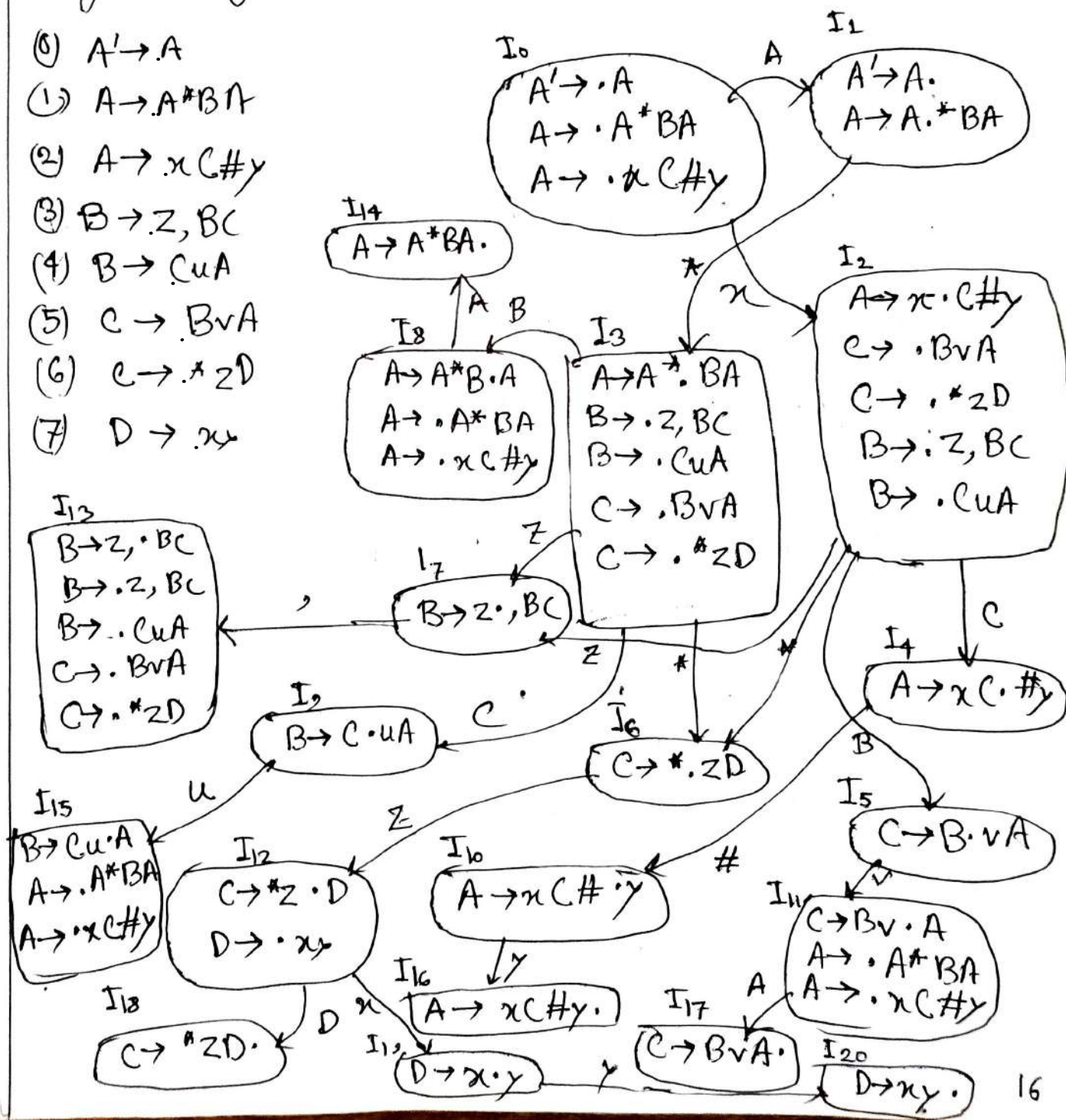
(3) $B \rightarrow z,BC$

(4) $B \rightarrow CuA$

(5) $C \rightarrow BvA$

(6) $C \rightarrow ^*zD$

(7) $D \rightarrow xy$



Action Table

States									
	x	y	z	#	#	,	u	v	\$
I ₀	S ₂								
I ₁				S ₃					Accepted
I ₂			S ₇	S ₆					
I ₃			S ₇	S ₆					
I ₄					S ₁₀				
I ₅								S ₁₁	
I ₆			S ₁₂						
I ₇						S ₁₃			
I ₈	S ₂								
I ₉							S ₁₅		
I ₁₀		S ₁₆							
I ₁₁	S ₂								
I ₁₂	S ₁₉								
I ₁₃									
I ₁₄	R ₁	R ₁	R ₁	R ₁	R ₁	R ₁	R ₁	R ₁	R ₁
I ₁₅									
I ₁₆	R ₂	R ₂	R ₂	R ₂	R ₂	R ₂	R ₂	R ₂	R ₂
I ₁₇	R ₅	R ₅	R ₅	R ₅	R ₅	R ₅	R ₅	R ₅	R ₅
I ₁₈	R ₆	R ₆	R ₆	R ₆	R ₆	R ₆	R ₆	R ₆	R ₆
I ₁₉									
I ₂₀	R ₇	R ₇	R ₇	R ₇	R ₇	R ₇	R ₇	R ₇	R ₇

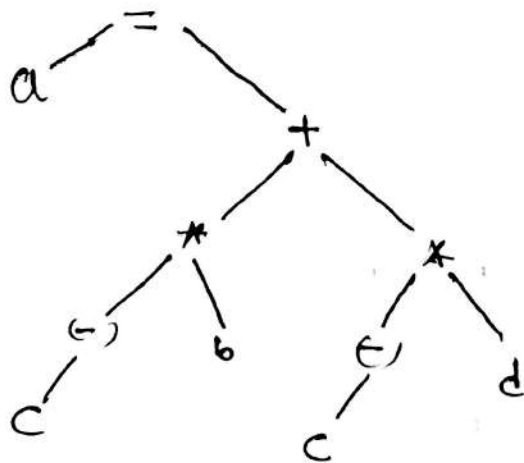
GOTO Table

State	A	B	c	D
I_0	1			
I_2		5	4	
I_3		8	9	
I_6				18
I_8	14			
I_{11}	17			
I_{12}				18

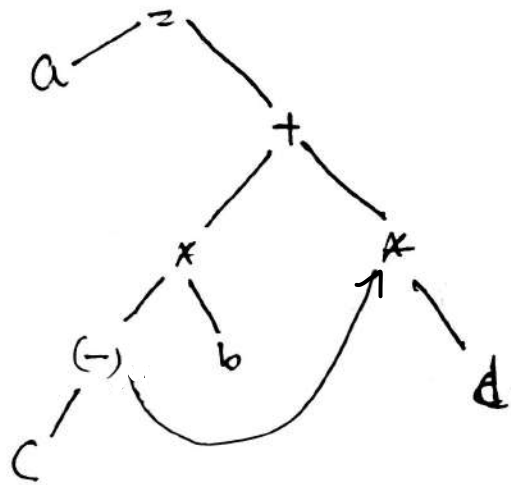
Answer to the question no: 13

Given expression: $a = (-c * b) + (-c * d)$

(i) Syntax tree



ii. Directed Acyclic Graph (DAG)



iii. Three address code

$$t_1 = -c$$

$$t_2 = t_1 * b$$

$$t_3 = t_1 * d$$

$$t_4 = t_2 + t_3$$

$$a = t_4$$

iv. Triple

#	Operator	Arg 1	Arg 2
0	-	c	-
1	*	(0)	b
2	*	(0)	d
3	+	(1)	(2)
4	=	(3)	a

Answer to the question no: 14

Given CFG

$S \rightarrow PART$

$P \rightarrow \text{om}$

$R \rightarrow \text{re} / \text{er} / \text{icr} / \text{Ra} / \epsilon$

$Q \rightarrow \text{a} / \text{e} / \text{i} / \text{o} / \text{u} / \epsilon$

$T \rightarrow \text{m} / \text{n}$

i. input string = omicron

LMD

$S \rightarrow PART$

$\rightarrow \text{om}QRT$

$\rightarrow \text{omi}RT$

$\rightarrow \text{omi}RQT$

$\rightarrow \text{omicr}QT$

$\rightarrow \text{omicr}OT$

$\rightarrow \text{omicron}$

RMD

$S \rightarrow PART$

$\rightarrow PQRn$

$\rightarrow PQRQn$

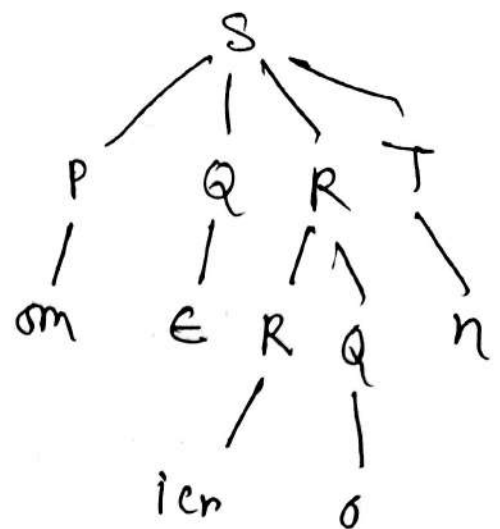
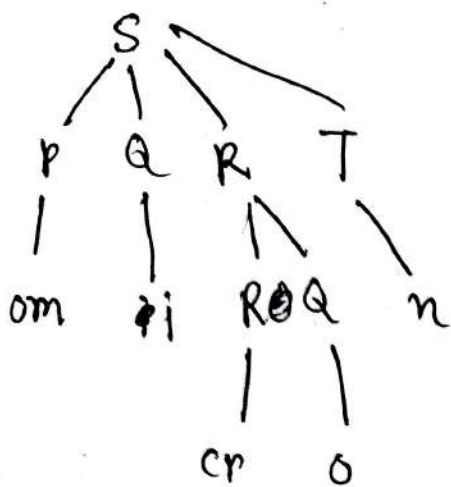
$\rightarrow PQRon$

$\rightarrow PQ\text{ per}on$

$\rightarrow P\text{icr}on$

$\rightarrow \text{omicron}$

ii. The above mentioned grammar is ambiguous because the rule $R \rightarrow RQ / \epsilon$ allows multiple ways to group vowels Q with R, which can lead to different parse trees for the same string.



So, the CFG is ambiguous.

Answer to the question no: 15

i) General leader selection rules for intermediate code generation —

Rule 1: The first statement of the program is a leader

Rule 2: Any statement that is the target of a jump. (goto, if goto) is a leader

Rule 3: Any statement that immediately follows a jump instruction is a leader.

ii) Leader selection by rule 2:

2, 7, 9, 12, 13, 19, 24, 27, 35, 36, 39

iii) Leader instructions by rule 3

5, 10, 11, 14, 15, 19, 24, 28, 30, 36, 40, 41, 46, 47, 50

iv) Final Leader Set

1, 2, 5, 7, 9, 10, 11, 12, 13, 14, 15, 19, 24, 27, 28, 30,
35, 36, 39, 40, 41, 46, 47, 50

Number of basic blocks = Number of leaders = 24

v) ~~Yes~~^{No}, some lines, 19, 24, 36 etc appear
as jump targets (rule 2), and after a jump
(rule 3)

They are selected more than one rule, but no
instruction is selected more than twice.

Answer to the question no: 16

Given expression -

$$((rate \times (vat + tan)) - ((cost * tan) + (revenue + (vat + tan))))$$

a) Three address code

$$t_1 = vat + tan$$

$$t_2 = rate * t_1$$

$$t_3 = cost * tan$$

$$t_4 = vat + tan$$

$$t_5 = revenue + t_4$$

$$t_6 = t_3 + t_5$$

$$t_7 = t_2 - t_6$$

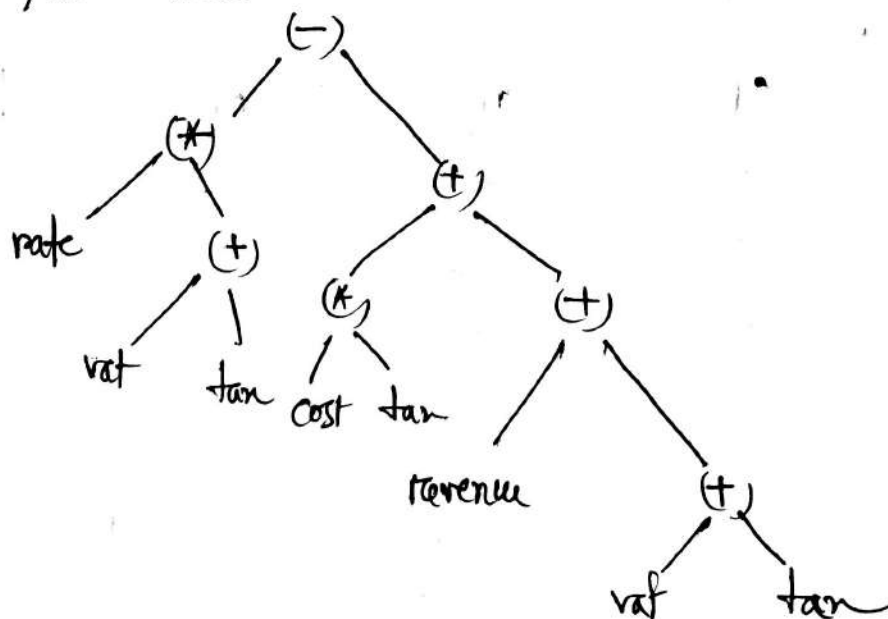
b) Quadruples

#	op	arg1	arg2	arg3
0	+	vat	tan	t ₁
1	*	rate	t ₁	t ₂
2	*	cost	tan	t ₃
3	+	vat	tan	t ₄
4	+	revenue	t ₄	t ₅
5	+	t ₃	t ₅	t ₆
6	-	t ₂	t ₆	t ₇

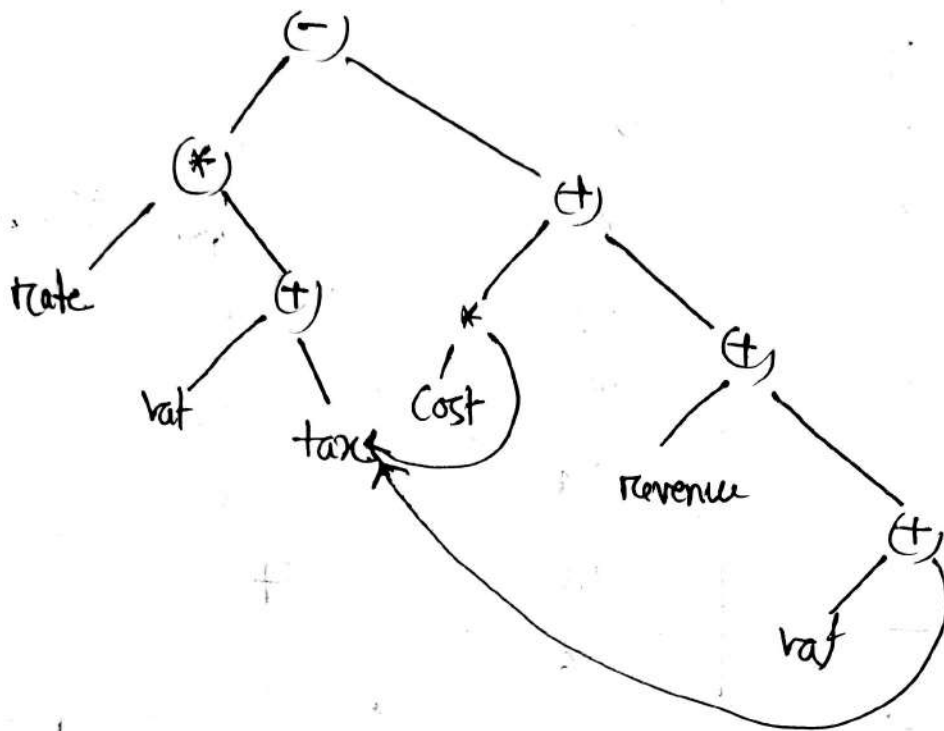
c) Triples

#	Op	arg 1	arg 2
0	+	vat	tax
1	*	rate	(0)
2	*	cost	tax
3	+	vat	tax
4	+	revenue	(3)
5	+	(2)	(4)
6	-	(1)	(5)

d) ~~Directed Acyclic Graph (DAG)~~ Syntax Tree



e) Directed Acyclic Graph (DAG)



Answer to the question no: 17

$$x + x * (y - z) + (y - z) * p + q - x + z$$

a) Three Address Code

$$t_1 = y - z$$

$$t_2 = x * t_1$$

$$t_3 = t_1 + p$$

$$t_4 = x + t_2$$

$$t_5 = t_4 + t_3$$

$$t_6 = t_5 + q$$

$$t_7 = t_6 - x$$

$$t_8 = t_7 + z$$

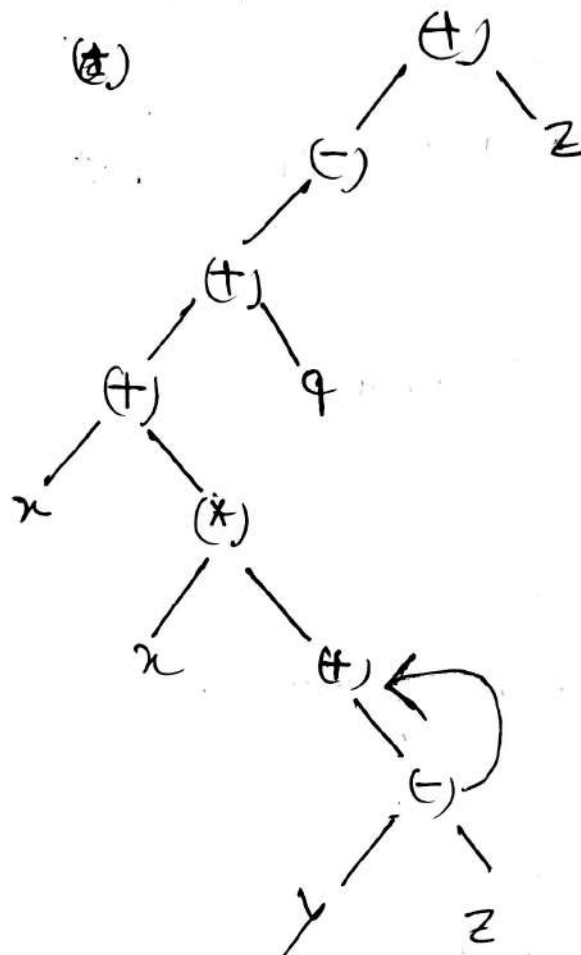
b) Quadruples

#	op	arg 1	arg 2	arg 3
0	-	r	z	t ₁
1	*	r	t ₁	t ₂
2	*	t ₁	p	t ₃
3	+	r	t ₂	t ₄
4	+	t ₄	t ₃	t ₅
5	+	t ₅	q	t ₆
6	-	t ₆	r	t ₇
7	+	t ₇	z	t ₈

c) Indirect Triples

#	Statement	#	op	arg 1	arg 2
01	21	0	-	r	z
1	22	1	*	r	(0)
3	23	2	*	(0)	p
4	24	3	+	r	(1)
5	25	4	+	(3)	(2)
6	26	5	+	(4)	q
7	27	6	-	(5)	r
8		7	+	(6)	z

1) Directed Acyclic Graph (DAG)



Answer to the question no: 18

a. The code optimization techniques that can be applied to the above code.

i) Constant Folding: compile time evaluation of constant expression

$$a=10, b=2$$

$$a+b=12$$

$$a*b=20$$

2. Common Subexpression Elimination

Avoid computing the same expression

$a+b$ - computed twice

$a+b$ - computed inside loop

3. Copy Propagation

Replace variables with their known values

$i = f \rightarrow i = 12$

4. Dead Code Elimination

Remove code that never executes

$\text{if}(c < 0)$, it's always false ($c = 12$)

k_2 never used

temp2 is undefined

b) Optimized Code

```
int main() {  
    int c = 12;  
    int f = 12;  
    int i = 12;  
    int k2 = 20;  
    while (i < 20) {  
        i++;  
    }  
    return i;  
}
```


Answer to the question no: 19

a) Copy Propagation: Copy propagation is an optimization technique where unnecessary copy assignments are eliminated by replacing the copied variable with its original value.

Example 1

Before copy propagation

$a = b;$

$\text{supan} = a + 5;$

$\text{roy} = a * 2;$

here, a is just a copy of b

After copy propagation

$\text{supan} = b + 5;$

$\text{roy} = b * 2;$

b) Strength Reduction: It replaces expensive operations with cheaper operations inside loops.

Example: Before strength reduction

```
for (i = 0; i < n; i++) {  
     $x = i * 4;$   
}
```

After strength reduction

```
x = 0;  
for (i = 0; i < n; i++) {  
    x = x + 4;  
}
```

Answer to the question no: 20

Constant propagation is a code optimization technique where the compiler replaces variables whose values are known at compile time with their constant values throughout the program.

It detects statements where a variable is assigned a constant value

$a = 5;$

Then, substitute the variable with its constant value where it appears

$b = a + 3; \rightarrow b = 5 + 3$

If the variable is reassigned later, it is removed from the constant table

$a = 10;$

Answer to the question no: 21

Dead Code Elimination,

1. $a = b + 1$
2. $b = c * 2$
3. $d = b - a$
4. $b = 5$
5. $\text{output}(d)$

Steps:

1. Statement 5 uses variable d , so statement 3 is required.
2. Statement 3 uses variables b and a , so statements 1 and 2 are required.
3. Statement 4 ($b = 5$):
This assignment to b happens after d is computed.
 $b = 5$ is never used anywhere later.
4. Statement 4 is dead code.

Code after dead code elimination

1. $a = b + 1$
2. $b = c * 2$
3. $d = b - a$
5. $\text{output}(d)$

Answer to the question no: 22,

⇒ Difference between Local Optimization and Global Optimization.

(i) Local Optimization: Local optimization is applied within a small section of code, usually a basic block.

Example:

$$a = 3 + 5$$

$$b = a * 2$$

after local optimization:

$$a = 8$$

$$b = 8 * 2$$

② Global Optimization: Global Optimization is applied across multiple basic blocks or the entire program, considering control flow and data flow.

Example:

$$a = b + c$$

$$d = b + c$$

After global optimization

$$a = b + c$$

$$d = a$$